

Exponential Backoff with Jitter, Worked Out by Hand

Naive constant retries cause thundering-herd outages. We compute the exact backoff schedule, derive why jitter de-synchronises clients, and quantify the collision it prevents.

What this document does. When a call fails, retrying badly is worse than not retrying. This document uses the course's exact formula $\text{delay} = \min(\text{cap}, \text{base} \cdot 2^{\text{attempt}}) \cdot \text{rand}(0.5, 1.5)$ to compute a full retry schedule, derives the geometric total wait, and shows numerically how jitter turns a synchronised stampede into a spread-out trickle.

Concepts covered: exponential growth 2^{attempt} · the cap · geometric-series total wait · jitter as a uniform multiplier · the thundering herd · collision probability after de-correlation.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: the formula

The resilient retry delay is

$$\text{delay}(a) = \min(\text{cap}, \text{base} \cdot 2^a) \cdot \text{rand}(0.5, 1.5),$$

with attempt index $a = 0, 1, 2, \dots$, $\text{base} = 200$ ms, and $\text{cap} = 10,000$ ms. The two pieces do different jobs: $\text{base} \cdot 2^a$ is the *exponential* spread over time; $\text{rand}(0.5, 1.5)$ is the *jitter* that de-synchronises clients.

Intuition

When a server recovers from an outage, every client that was waiting hits it at once — and knocks it back down. Exponential backoff spreads each client's retries further apart over time; jitter spreads *different clients* apart from each other at the same instant. You need both: exponential alone still has all clients retrying in lockstep at $t = 200, 400, 800, \dots$

2 Step 1 — The exponential schedule

The base delay $\text{base} \cdot 2^a$ doubles each attempt until it hits the cap:

attempt a	$200 \cdot 2^a$ (ms)	after cap 10,000
0	200	200
1	400	400
2	800	800
3	1,600	1,600
4	3,200	3,200
5	6,400	6,400
6	12,800	10,000 (capped)

Intuition

The cap stops the delay running away to minutes or hours. Without it, attempt 12 would wait $200 \cdot 2^{12} = 819 \text{ s} \approx 14 \text{ minutes}$ — almost certainly past any useful deadline. The cap turns the tail from exponential into a constant “poll every 10s” once the doubling has done its spreading work.

3 Step 2 — Total wait is a geometric series

Ignoring jitter (mean 1) and the cap, the total time across n retries is a geometric series:

$$\sum_{a=0}^{n-1} \text{base} \cdot 2^a = \text{base} (2^n - 1).$$

For $n = 6$ uncapped: $200 (2^6 - 1) = 200 \cdot 63 = 12,600 \text{ ms} = 12.6 \text{ s}$. With the cap applied at $a = 6$ and one more capped attempt, the realised schedule $200+400+800+1600+3200+6400+10000 = 22,600 \text{ ms} \approx 22.6 \text{ s}$ across 7 retries.

Watch out

Exponential totals balloon fast: $n = 10$ uncapped is $200(2^{10} - 1) = 204,600 \text{ ms} \approx 3.4 \text{ minutes}$. Choose max-attempts and cap together against your deadline — “retry forever with backoff” is how a transient blip becomes a multi-minute hang the user blames on you.

4 Step 3 — Jitter: the uniform multiplier

Jitter multiplies each delay by $U \sim \text{Uniform}(0.5, 1.5)$. Its statistics:

$$\mathbb{E}[U] = 1.0, \quad \text{Var}(U) = \frac{(1.5 - 0.5)^2}{12} = \frac{1}{12} = 0.0833, \quad \sigma_U = 0.289.$$

So jitter does not change the *expected* delay (still $\text{base} \cdot 2^a$), but it scatters each client uniformly across a window of width $(1.5 - 0.5) \cdot \text{base} \cdot 2^a = \text{base} \cdot 2^a$ centred on the nominal delay. At attempt 3 that window is 1,600 ms wide.

5 Step 4 — The thundering herd, quantified

Suppose $K = 1,000$ clients all failed at the same instant and all retry at attempt 3 (nominal 1,600 ms).

Without jitter: all 1,000 fire at exactly $t = 1,600 \text{ ms}$ — a 1,000-wide spike the recovering server must absorb in one tick.

With jitter: they spread uniformly over a window of width $W = 1,600 \text{ ms}$. The expected arrival rate is

$$\lambda = \frac{K}{W} = \frac{1000}{1600 \text{ ms}} = 0.625 \text{ clients/ms},$$

so in any 1 ms tick the expected load is 0.625 clients instead of 1,000 — a 1600 $\times$ reduction in peak concurrency. The same retries, spread out, stop looking like an attack.

Mini-example — this operation in isolation

Probability two specific clients still collide within a $\Delta = 1 \text{ ms}$ tick after jittering over $W = 1600 \text{ ms}$: each lands uniformly, so $P(|t_1 - t_2| < \Delta) \approx 2\Delta/W = 2/1600 = 0.00125$. Pairwise collisions become rare; the herd dissolves.

6 The argument, at a glance

#	Idea	Formula	Value
1	Exponential delay	$\text{base} \cdot 2^a$	doubles
1	Cap	$\min(\text{cap}, \cdot)$	10,000 ms
2	Total wait	$\text{base}(2^n - 1)$	22.6 s ($n=7$, capped)
3	Jitter mean / var	1.0 / 1/12	$\sigma = 0.289$
4	Peak-load drop	$K \rightarrow K/W$ per ms	1600×
4	Pairwise collision	$2\Delta/W$	0.00125

7 Equation cheat-sheet

Backoff delay	$\min(\text{cap}, \text{base} \cdot 2^a) \cdot U, U \sim \text{Unif}(0.5, 1.5)$
Total wait (uncapped)	$\text{base}(2^n - 1)$
Jitter mean / variance	$\mathbb{E}[U] = 1, \text{Var} = 1/12$
Spread window width	$(1.5 - 0.5) \text{base } 2^a$
Arrival rate (jittered)	$\lambda = K/W$
Pairwise collision	$\approx 2\Delta/W$

A Reference implementation

```
def base_delay(a, base=200, cap=10_000):
    return min(cap, base * 2**a)

sched = [base_delay(a) for a in range(7)]
print(sched)                # [200,400,800,1600,3200,6400,10000]
print(sum(sched))           # 22600 ms total

# herd: K clients over window W -> peak per ms
K, W = 1000, 1600
print(K / W)                # 0.625 clients/ms
print(2 * 1 / W)            # 0.00125 pairwise collision
```

— end of document —